

EXPERIMENT – 11.2

Name: K. ANSHU

Roll No: 2520030440

Title

Multithreaded Counter Application using POSIX Threads and Mutex Synchronization

Aim

To modify the multithreaded counter application using a mutex and compare the synchronized result with the race-condition version.

Theory

A mutex (mutual exclusion) is a synchronization mechanism used to protect shared resources from simultaneous access by multiple threads. `pthread_mutex_lock()` locks the critical section before modifying the shared counter, while `pthread_mutex_unlock()` releases the lock after the modification. This prevents simultaneous updates to the shared counter.

Algorithm

1. Initialize the shared counter to zero.
2. Initialize a mutex using `pthread_mutex_init()`.
3. Create four threads using `pthread_create()`.
4. Each thread locks the mutex before updating the counter.
5. Increment the counter 100000 times.
6. Unlock the mutex after each update.
7. Use `pthread_join()` to wait for all threads.
8. Display the expected and actual counter values.
9. Destroy the mutex using `pthread_mutex_destroy()`.
10. Compare the result with the race-condition version.

Program Code

```
#include <stdio.h>
#include <pthread.h>

#define THREADS 4
#define ITERATIONS 100000

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg)
```

```

{
    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t threads[THREADS];

    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_create(&threads[i], NULL, increment, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);

    printf("Expected counter value: %d\n", THREADS * ITERATIONS);
    printf("Actual counter value: %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}

```

Execution and Screenshots

Screenshot 1 – Program Code

```
admin@heeraj@DESKTOP-J39GHP7:~$  
admin@heeraj@DESKTOP-J39GHP7:~$ cat > exp11_mutex.c <<'EOF'  
#include <stdio.h>  
#include <pthread.h>  
  
#define THREADS 4  
#define ITERATIONS 100000  
  
int counter = 0;  
pthread_mutex_t lock;  
  
void *increment(void *arg)  
{  
    for (int i = 0; i < ITERATIONS; i++)  
    {  
        pthread_mutex_lock(&lock);  
        counter++;  
        pthread_mutex_unlock(&lock);  
    }  
  
    return NULL;  
}  
  
int main()  
{  
    pthread_t threads[THREADS];  
  
    pthread_mutex_init(&lock, NULL);  
  
    for (int i = 0; i < THREADS; i++)
```

Screenshot 2 – Compilation

```

        return NULL;
    }

int main()
{
    pthread_t threads[THREADS];

    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_create(&threads[i], NULL, increment, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);

    printf("Expected counter value: %d\n", THREADS * ITERATIONS);
    printf("Actual counter value: %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}
EOF

```

Screenshot 3 – Execution Output

```

admin@heeraj@DESKTOP-J39GHP7:~$ gcc exp11_mutex.c -o exp11_mutex -pthread
admin@heeraj@DESKTOP-J39GHP7:~$ ./exp11_mutex
Expected counter value: 400000
Actual counter value: 400000
admin@heeraj@DESKTOP-J39GHP7:~$ |

```

Observed Output

```

Expected counter value: 400000
Actual counter value: 400000

```

Result

The multithreaded counter application was successfully synchronized using a mutex. The expected counter value was 400000 and the actual counter value was also 400000, demonstrating that mutex synchronization prevents the race condition.